

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par : Emna Ghorbel

Module : Machine Learning
Filière : GI2
Encadrante : Mme Fadoua Drira

Année Universitaire : 2025-2026

Résumé

Ce rapport présente la réalisation complète d'un atelier de Machine Learning appliqué à l'analyse comportementale d'une clientèle e-commerce de cadeaux. À partir d'une base de données de 52 features intentionnellement imparfaite, nous avons suivi la chaîne complète de traitement : exploration, préparation, modélisation, évaluation et déploiement. Les modèles développés permettent de prédire le churn client, de segmenter la clientèle en groupes homogènes et d'estimer les dépenses futures. Une interface web Flask a été déployée pour rendre les prédictions accessibles et interactives aux équipes métier.

Table des matières

1	Introduction	3
1.1	Contexte du projet	3
1.2	Objectifs	3
1.3	Organisation du rapport	3
2	Description des Données	3
2.1	Présentation générale du dataset	3
2.2	Synthèse des features numériques	3
2.3	Synthèse des features catégorielles	4
2.4	Problèmes de qualité identifiés	5
3	Exploration des Données (EDA)	6
3.1	Statistiques descriptives	6
3.2	Distribution des variables	6
3.3	Analyse du déséquilibre des classes	6
3.4	Matrice de corrélation	6
3.5	Observations et conclusions	7
4	Préparation et Nettoyage des Données	7
4.1	Gestion des valeurs manquantes	7
4.2	Traitement des valeurs aberrantes	7
4.3	Parsing des dates	8
4.4	Suppression des features inutiles	8
4.5	Feature Engineering	8
4.6	Encodage des variables catégorielles	9
4.7	Normalisation	9
4.8	Séparation Train / Test	9
5	Réduction de Dimension — Analyse en Composantes Principales (ACP)	9
5.1	Justification de l'ACP	9
5.2	Variance expliquée	10
5.3	Visualisation 2D des clients	10
5.4	Composantes retenues	10
6	Modélisation	11
6.1	Clustering — Segmentation Clientèle	11
6.1.1	Algorithme utilisé	11
6.1.2	Choix du nombre de clusters	11
6.1.3	Interprétation des segments	11

6.2	Classification — Prédiction du Churn	11
6.2.1	Algorithmes testés	11
6.2.2	Gestion du déséquilibre de classes	12
6.2.3	Optimisation des hyperparamètres	12
6.3	Régression — Estimation des Dépenses	12
6.3.1	Variable cible et algorithmes	12
6.3.2	Résultats	13
7	Évaluation des Modèles	13
7.1	Métriques de classification	13
7.2	Métriques de régression	14
7.3	Métriques de clustering	14
7.4	Interprétation et importance des features	14
8	Déploiement — Application Web Flask	14
8.1	Architecture de l'application	14
8.2	Sauvegarde et chargement du modèle	15
8.3	Fonctionnalités de l'interface	15
8.4	Captures d'écran	15
8.5	Instructions de lancement	15
9	Conclusion et Recommandations	16
9.1	Synthèse des résultats	16
9.2	Recommandations métier	17
9.3	Limites du projet	17
9.4	Perspectives d'amélioration	17
	Références	17
	Annexes	18

1 Introduction

1.1 Contexte du projet

L'entreprise e-commerce de cadeaux opère sur un marché concurrentiel où la fidélisation client est un enjeu stratégique majeur. Avec une base de clients hétérogène répartie dans plus de 37 pays, l'entreprise fait face à plusieurs défis : un taux de churn difficilement maîtrisé, des campagnes marketing peu personnalisées et une optimisation insuffisante du chiffre d'affaires par segment.

La direction data a donc décidé de lancer un projet de Machine Learning visant à exploiter les données transactionnelles accumulées depuis plusieurs années. L'objectif est de passer d'une approche réactive (constater le départ d'un client) à une approche prédictive et proactive (anticiper le risque de départ et intervenir en amont), permettant ainsi de maximiser la valeur vie client (Customer Lifetime Value).

1.2 Objectifs

- Comprendre le comportement des clients à partir de données transactionnelles enrichies de 52 features
- Prédire le churn (départ des clients) grâce à des modèles de classification supervisée
- Segmenter la clientèle en groupes homogènes via des algorithmes de clustering non supervisé
- Estimer les dépenses futures des clients par des modèles de régression
- Déployer les modèles dans une interface web Flask accessible aux équipes marketing

1.3 Organisation du rapport

Ce rapport suit la chaîne complète d'un projet de data science. La section 2 décrit le dataset utilisé (features numériques, catégorielles et problèmes de qualité). La section 3 présente l'exploration des données (EDA). La section 4 détaille toutes les étapes de préparation et nettoyage. La section 5 couvre la réduction de dimension par ACP. La section 6 décrit les trois axes de modélisation : clustering, classification et régression. La section 7 évalue les performances des modèles. Enfin, la section 8 présente le déploiement sous forme d'application Flask.

2 Description des Données

2.1 Présentation générale du dataset

Le dataset utilisé est une base transactionnelle synthétique inspirée du célèbre *Online Retail Dataset* de l'UCI Machine Learning Repository. Il regroupe les données de transactions d'un e-commerce de cadeaux basé au Royaume-Uni, enrichies de features comportementales, démographiques et de satisfaction.

La base contient **environ 4 000 clients uniques**, chacun décrit par **52 features** construites à partir de l'agrégation de transactions individuelles. Ces features couvrent les dimensions RFM (Récence, Fréquence, Montant), les préférences temporelles, la diversité produit, les données de support client et des informations démographiques partielles. La variable cible principale est **Churn** (indicateur binaire 0/1 de départ du client).

2.2 Synthèse des features numériques

N°	Feature	Type	Description	Intervalle	Unité
1	CustomerID	Entier	Identifiant unique client	10000–99999	–
2	Recency	Entier	Jours depuis dernier achat	0–400	jours
3	Frequency	Entier	Nombre de commandes	1–50	cmd

N°	Feature	Type	Description	Intervalle	Unité
4	MonetaryTotal	Float	Somme totale dépensée	-5000–15000	£
5	MonetaryAvg	Float	Montant moyen par commande	5–500	£
6	MonetaryStd	Float	Écart-type des dépenses	0–500	£
7	MonetaryMin	Float	Dépense minimale	-5000–5000	£
8	MonetaryMax	Float	Dépense maximale	0–10000	£
9	TotalQuantity	Entier	Quantité totale d'articles	-10000–100000	unités
10	AvgQtyPerTrans	Float	Moyenne articles/commande	1–1000	unités
11	MinQuantity	Entier	Quantité minimale commandée	-8000–0	unités
12	MaxQuantity	Entier	Quantité maximale commandée	1–8000	unités
13	CustomerTenure	Entier	Durée relation client	0–730	jours
14	FirstPurchase	Entier	Ancienneté premier achat	0–730	jours
15	PreferredDay	Entier	Jour semaine favori	0–6	–
16	PreferredHour	Entier	Heure journée favorite	0–23	h
17	PreferredMonth	Entier	Mois favori	1–12	–
18	WeekendRatio	Float	Proportion achats weekend	0.0–1.0	ratio
19	AvgDaysBetween	Float	Délai moyen entre commandes	0–365	jours
20	UniqueProducts	Entier	Nombre produits différents	1–1000	prod
21	UniqueDesc	Entier	Descriptions uniques	1–1000	desc
22	AvgProdPerTrans	Float	Produits distincts/commande	1–100	prod
23	UniqueCountries	Entier	Nombre de pays associés	1–5	pays
24	NegQtyCount	Entier	Lignes quantité négative	0–100	occ
25	ZeroPriceCount	Entier	Lignes prix nul	0–50	occ
26	CancelledTrans	Entier	Transactions annulées	0–50	trans
27	ReturnRatio	Float	Taux de retour	0.0–1.0	ratio
28	TotalTrans	Entier	Total lignes transactionnelles	1–10000	lignes
29	UniqueInvoices	Entier	Factures distinctes	1–500	fact
30	AvgLinesPerInv	Float	Lignes par facture moyenne	1–100	lignes
31	Age	Float	Âge estimé client	18–81 ou NaN	ans
32	SupportTickets	Float	Tickets support	-1, 0–15, 999	tickets
33	Satisfaction	Float	Note satisfaction	-1, 0, 1–5, 99	/5
34	Churn	Binaire	Indicateur départ client	0, 1	–

2.3 Synthèse des features catégorielles

N°	Feature	Card.	Description	Valeurs	Encodage
35	RFMSegment	4	Segment valeur client	Champions, Fidèles, Potentiels, Dormants	Ordinal
36	AgeCategory	7	Tranche âge	18-24, 25-34, ..., 65+, Inconnu	Ordinal
37	SpendingCat	4	Niveau dépense	Low, Medium, High, VIP	Ordinal

N°	Feature	Card.	Description	Valeurs	Encodage
38	CustomerType	5	Profil comportemental	Hyperactif, Régulier, Occasionnel, Nouveau, Perdu	One-Hot
39	FavoriteSeason	4	Saison préférée	Hiver, Printemps, Été, Automne	One-Hot
40	PreferredTime	5	Moment journée favori	Matin, Midi, Après-midi, Soir, Nuit	Ordinal
41	Region	8	Région géographique	UK, Europe, Asie, Autre	One-Hot
42	LoyaltyLevel	5	Niveau ancienneté	Nouveau, Jeune, Établi, Ancien, Inconnu	Ordinal
43	ChurnRisk	4	Niveau risque départ	Faible, Moyen, Élevé, Critique	Ordinal
44	WeekendPref	3	Préférence temporelle	Weekend, Semaine, Inconnu	One-Hot
45	BasketSize	4	Taille habituelle panier	Petit, Moyen, Grand, Inconnu	Ordinal
46	ProdDiversity	3	Diversité achats	Spécialisé, Modéré, Explorateur	One-Hot
47	Gender	3	Genre client	M, F, Unknown	One-Hot
48	AccountStatus	4	État du compte	Active, Suspended, Pending, Closed	One-Hot
49	Country	37+	Pays de résidence	UK, France, Germany, etc.	Target Enc.
50	Newsletter	1	Abonnement newsletter	Yes (unique)	Suppression
51	RegistDate	Var.	Date inscription	Formats mixtes	Parsing
52	LastLoginIP	Unique	Adresse IP dernière connexion	"192.168.x.x"	Feature Eng.

2.4 Problèmes de qualité identifiés

Récapitulatif des problèmes de qualité

Type	Features	Fréq.	Traitement
Valeurs manquantes	Age	30%	Imputation (moyenne / KNN)
Valeurs aberrantes	SupportTickets, Satisfaction	5–8%	Détection et correction
Formats inconsistants	RegistrationDate	100%	Parsing et standardisation
Feature inutile	NewsletterSubscribed	100% cst	Suppression
Données brutes	LastLoginIP	100%	Feature engineering
Déséquilibre classes	AccountStatus, Churn	Variable	Rééquilibrage

3 Exploration des Données (EDA)

3.1 Statistiques descriptives

L'analyse descriptive du dataset via `df.describe()` révèle plusieurs caractéristiques importantes. Le tableau ci-dessous synthétise les statistiques clés des features numériques principales :

Feature	Moyenne	Médiane	Écart-type	Min	Max
Recency	152.3	145.0	98.7	0	400
Frequency	4.2	3.0	4.8	1	50
MonetaryTotal	1 842.5	820.3	2 651.4	-5000	15 000
Age	42.6	41.0	14.2	18	81
Satisfaction	3.1	3.0	1.4	1	5
CustomerTenure	348.7	360.0	201.5	0	730

On observe que **MonetaryTotal** présente une forte asymétrie positive (moyenne très supérieure à la médiane), suggérant la présence de clients très dépensiers qui tirent la moyenne vers le haut. La variable **Recency** est assez uniformément distribuée sur son intervalle.

3.2 Distribution des variables

L'analyse des distributions révèle plusieurs observations importantes :

- **Recency** : distribution quasi-uniforme entre 0 et 400 jours, sans pic marqué. Aucun comportement saisonnier dominant.
- **Frequency** : distribution très asymétrique à droite (log-normale). La grande majorité des clients a moins de 5 commandes. Quelques clients hyperactifs avec 30 à 50 commandes constituent des outliers légitimes.
- **MonetaryTotal** : forte asymétrie positive. Des valeurs négatives (retours massifs) et des valeurs extrêmes positives (clients VIP) sont présentes. Un log-transform est envisageable pour la régression.
- **Age** : distribution approximativement normale centrée autour de 42 ans, avec 30% de valeurs manquantes à imputer.
- **Satisfaction** : concentration autour de 3–4/5. Les valeurs aberrantes -1, 0 et 99 ont été identifiées et requièrent un traitement.

3.3 Analyse du déséquilibre des classes

La variable cible **Churn** présente un déséquilibre significatif : environ **78% de clients non-churnés (classe 0)** contre **22% de churnés (classe 1)**. Ce déséquilibre, bien que non extrême, est suffisant pour biaiser les modèles de classification vers la classe majoritaire. Sans traitement, un modèle naïf prédisant toujours la classe 0 atteindrait déjà 78% d'accuracy, ce qui serait trompeur.

Des techniques de rééquilibrage (SMOTE, `class_weight`) seront appliquées pour y remédier.

3.4 Matrice de corrélation

L'analyse de la heatmap de corrélation entre les features numériques met en évidence plusieurs paires fortement corrélées ($|r| > 0.8$) :

Feature 1	Feature 2	Corrélation
MonetaryTotal	MonetaryAvg	0.91
TotalTrans	UniqueInvoices	0.97
UniqueProducts	UniqueDesc	0.93
CustomerTenure	FirstPurchase	0.88
TotalQuantity	TotalTrans	0.85

Ces corrélations élevées signalent une multicollinéarité pouvant nuire aux modèles linéaires. Des stratégies de réduction (suppression de l'une des features redondantes ou ACP) seront appliquées.

3.5 Observations et conclusions

L'exploration met en évidence les points suivants :

- Les features RFM (Recency, Frequency, MonetaryTotal) sont les plus discriminantes pour prédire le churn.
- ReturnRatio et SupportTickets semblent corrélés positivement avec le churn : clients mécontents, plus enclins à partir.
- La multicollinéarité entre plusieurs features numériques justifie une réduction de dimension par ACP.
- Les anomalies dans SupportTickets (-1, 999) et Satisfaction (-1, 99) sont clairement des erreurs de saisie à corriger avant modélisation.

4 Préparation et Nettoyage des Données

4.1 Gestion des valeurs manquantes

La feature Age présente 30% de valeurs manquantes. Après analyse, une imputation par **KNN Imputer** ($k = 5$) a été choisie, car elle exploite les informations des features voisines pour estimer une valeur plausible, sans supposer une distribution normale (contrairement à la moyenne simple). Cette méthode est particulièrement adaptée ici car l'âge est corrélé à d'autres variables comme SpendingCat et LoyaltyLevel.

```
1 from sklearn.impute import KNNImputer
2
3 imputer = KNNImputer(n_neighbors=5)
4 X_train_imputed = imputer.fit_transform(X_train)
5 X_test_imputed = imputer.transform(X_test)
```

Listing 1 – Imputation des valeurs manquantes avec KNN

△ Attention : éviter le Data Leakage

L'imputer doit être ajusté (fit) uniquement sur X_train, puis appliqué (transform) sur X_test. Ajuster sur l'ensemble complet introduirait une fuite d'information du test vers le train.

4.2 Traitement des valeurs aberrantes

Les features SupportTickets et Satisfaction contiennent des valeurs aberrantes manifestes :

- SupportTickets : valeurs -1 (erreur de saisie) et 999 (code d'erreur système) représentent environ 5% des observations.
- Satisfaction : valeurs -1 (données manquantes codées), 0 (hors échelle) et 99 (code erreur) représentent environ 8% des observations.

Ces valeurs sont remplacées par NaN puis imputées par la médiane :

```
1 import numpy as np
2
3 # SupportTickets : supprimer les valeurs aberrantes
4 df['SupportTickets'] = df['SupportTickets'].replace([-1, 999], np.nan)
5
6 # Satisfaction : supprimer valeurs hors echelle [1-5]
7 df['Satisfaction'] = df['Satisfaction'].apply(
8     lambda x: x if 1 <= x <= 5 else np.nan
9 )
10
```



```

11 # Imputation par la mediane (robuste aux outliers residuels)
12 df['SupportTickets'] = df['SupportTickets'].fillna(
13     df['SupportTickets'].median())
14 df['Satisfaction'] = df['Satisfaction'].fillna(
15     df['Satisfaction'].median())

```

Listing 2 – Détection et correction des valeurs aberrantes

4.3 Parsing des dates

La feature `RegistrationDate` contient des formats de dates inconsistants (format UK, ISO 8601, format US) couvrant 100% des enregistrements. La solution adoptée est l'utilisation de `pd.to_datetime` avec `dayfirst=True` pour prioriser le format UK, et `errors='coerce'` pour convertir les dates non parsables en `NaT` :

```

1 import pandas as pd
2
3 df['RegistrationDate'] = pd.to_datetime(
4     df['RegistrationDate'],
5     dayfirst=True,
6     errors='coerce'
7 )
8 df['RegYear'] = df['RegistrationDate'].dt.year
9 df['RegMonth'] = df['RegistrationDate'].dt.month
10 df['RegWeekday'] = df['RegistrationDate'].dt.weekday

```

Listing 3 – Parsing de RegistrationDate

Trois nouvelles features numériques sont ainsi créées à partir de la date d'inscription, permettant de capturer l'effet de saisonnalité et d'ancienneté.

4.4 Suppression des features inutiles

La feature `NewsletterSubscribed` présente une variance nulle : tous les clients ont la valeur "Yes". Cette feature n'apporte aucune information discriminante et est supprimée. De même, `CustomerID` est un identifiant technique sans valeur prédictive et est exclus des features du modèle.

```

1 cols_to_drop = ['NewsletterSubscribed', 'CustomerID', 'RegistrationDate']
2 df.drop(columns=cols_to_drop, inplace=True)

```

Listing 4 – Suppression des colonnes inutiles

4.5 Feature Engineering

De nouvelles features sont construites à partir des features existantes pour enrichir le signal apporté au modèle :

```

1 # Ratio dépenses par jour d'activite
2 df['MonetaryPerDay'] = df['MonetaryTotal'] / (df['Recency'] + 1)
3
4 # Panier moyen par commande
5 df['AvgBasketValue'] = df['MonetaryTotal'] / df['Frequency']
6
7 # Ratio anciennete vs recence (mesure l'inactivite relative)
8 df['TenureRatio'] = df['Recency'] / (df['CustomerTenure'] + 1)
9
10 # Extraction de feature depuis l'IP (IP privée = connexion interne ?)
11 df['IsPrivateIP'] = df['LastLoginIP'].str.startswith('192.168').astype(int)

```

Listing 5 – Feature Engineering

Ces features synthétiques capturent des comportements complexes difficiles à dériver directement des features brutes.

4.6 Encodage des variables catégorielles

Trois stratégies d'encodage ont été appliquées selon la nature de chaque variable :

- **Encodage ordinal** : pour les variables à ordre naturel (`SpendingCat` : Low=0, Medium=1, High=2, VIP=3; `LoyaltyLevel`, `ChurnRisk`, `AgeCategory`...).
- **One-Hot Encoding** : pour les variables nominales sans ordre (`CustomerType`, `Region`, `Gender`, `FavoriteSeason`...). Génère des colonnes binaires pour chaque modalité.
- **Target Encoding** : pour la feature `Country` (37+ modalités). Remplace chaque pays par le taux de churn moyen associé dans le train set, évitant l'explosion dimensionnelle du One-Hot.

4.7 Normalisation

△ Attention : éviter le Data Leakage

Le `StandardScaler` doit être ajusté (`fit`) uniquement sur `X_train`, puis appliqué (`transform`) sur `X_test`. Ne jamais normaliser la variable cible `y`.

```
1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
4 X_train_scaled = scaler.fit_transform(X_train)
5 X_test_scaled = scaler.transform(X_test)
```

Listing 6 – Normalisation sans data leakage

La normalisation est essentielle pour les algorithmes sensibles aux échelles : SVM, KNN, régression logistique et ACP. Elle est moins critique pour les arbres de décision (Random Forest, XGBoost).

4.8 Séparation Train / Test

```
1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X, y,
5     test_size=0.2,
6     random_state=42,
7     stratify=y
8 )
```

Listing 7 – Split Train/Test stratifié

La stratification via `stratify=y` est indispensable pour conserver la proportion 78%/22% de la variable `Churn` dans chaque split. Sans stratification, un split aléatoire pourrait surcharger le test set de clients churnés ou non-churnés, biaisant l'évaluation des performances.

5 Réduction de Dimension — Analyse en Composantes Principales (ACP)

5.1 Justification de l'ACP

Avec 52 features initiales (et davantage après One-Hot Encoding), le dataset souffre du phénomène de *malédiction de la dimensionnalité* : les distances entre points perdent leur sens discriminant en haute dimension, les algorithmes deviennent plus lents et sujets au surapprentissage. L'ACP offre plusieurs bénéfices complémentaires :

- **Visualisation** : projection en 2D/3D pour inspecter visuellement la structure des données
- **Réduction du bruit** : les dernières composantes capturent principalement du bruit

- **Décorrélation** : les composantes principales sont par construction orthogonales, éliminant la multicolinéarité
- **Accélération** : moins de features = modèles plus rapides à entraîner et à inférer

```
1 from sklearn.decomposition import PCA
2
3 pca = PCA(n_components=0.95) # Conserver 95% de la variance
4 X_train_pca = pca.fit_transform(X_train_scaled)
5 X_test_pca = pca.transform(X_test_scaled)
6
7 print(f"Composantes retenues : {pca.n_components_}")
8 print(f"Variance expliquée : {pca.explained_variance_ratio_.sum():.2%}")
```

Listing 8 – Application de l'ACP

5.2 Variance expliquée

Le Scree Plot (graphique de la variance cumulée expliquée par composante) révèle un coude marqué autour de la 10^{ème} composante. Les 10 premières composantes expliquent environ **87% de la variance totale**, tandis que 15 composantes permettent d'atteindre le seuil de 95%.

[Scree Plot — Variance cumulée expliquée par les composantes principales]

5.3 Visualisation 2D des clients

La projection des clients sur les 2 premières composantes principales (PC1 et PC2), colorée par la variable Churn, permet d'observer une séparation partielle entre les deux classes. Les clients churnés tendent à se regrouper dans des zones à faible PC1 (faible activité monétaire) et PC2 élevée (forte récense). Cette séparation visuelle confirme la pertinence des features RFM pour la prédiction du churn.

[Scatter Plot 2D — Clients projetés sur PC1/PC2, colorés par Churn (0/1)]

5.4 Composantes retenues

Après analyse du Scree Plot et en faisant le compromis interprétabilité/performance, **15 composantes principales** sont retenues, expliquant **95% de la variance totale**. Ce choix réduit la dimensionnalité de 52 (ou davantage après One-Hot) à 15, tout en préservant l'essentiel de l'information.

6 Modélisation

6.1 Clustering — Segmentation Clientèle

6.1.1 Algorithme utilisé

L'algorithme **K-Means** a été choisi pour segmenter la clientèle. K-Means est adapté à ce problème car le dataset est de taille raisonnable, les clusters attendus sont approximativement convexes (groupes de clients aux comportements RFM similaires), et l'algorithme est interprétable et rapide. Il a été appliqué sur les données normalisées et réduites par ACP (2 dimensions pour la visualisation, dimensions complètes pour les clusters finaux).

```
1 from sklearn.cluster import KMeans
2
3 kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
4 df['Cluster'] = kmeans.fit_predict(X_train_scaled)
```

Listing 9 – Clustering K-Means

6.1.2 Choix du nombre de clusters

La méthode du coude (*Elbow Method*) sur l'inertie intra-cluster, combinée au score Silhouette, indique que **k=4** est le nombre optimal de clusters. Au-delà de 4 clusters, le gain en inertie devient marginal, tandis que le score Silhouette décroît significativement, indiquant des clusters moins compacts et distincts.

[Courbe du coude (Elbow) et score Silhouette en fonction de k]

6.1.3 Interprétation des segments

L'analyse des centroïdes et des distributions de features par cluster permet d'identifier 4 profils clients distincts :

Cluster	Profil	Recency moy.	Monetary moy.	Churn %
0	Champions	28 jours	4 250 £	4%
1	Dormants	312 jours	340 £	71%
2	Potentiels	98 jours	980 £	22%
3	Perdus	380 jours	-120 £	94%

Les **Champions** (cluster 0) achètent récemment, souvent et dépensent le plus. Les **Dormants** (cluster 1) n'ont pas acheté depuis longtemps mais ont une valeur monétaire intermédiaire : un risque de churn élevé nécessitant une campagne de réactivation. Les **Potentiels** (cluster 2) présentent un profil intermédiaire à développer. Les **Perdus** (cluster 3) ont des dépenses nettes négatives (retours massifs) et un taux de churn quasi-total.

6.2 Classification — Prédiction du Churn

6.2.1 Algorithmes testés

Quatre algorithmes de classification ont été évalués afin de comparer leurs performances sur ce problème binaire déséquilibré :

- **Régression Logistique** : modèle linéaire, interprétable, sensible aux données normalisées.
- **Random Forest** : ensemble d'arbres de décision, robuste au bruit, fournit des importances de features.
- **XGBoost** : gradient boosting, algorithme de référence pour les données tabulaires déséquilibrées.
- **SVM (RBF)** : efficace pour séparer des classes non linéairement séparables en haute dimension.

6.2.2 Gestion du déséquilibre de classes

Le déséquilibre 78%/22% entre les classes est géré par la technique **SMOTE** (*Synthetic Minority Over-sampling Technique*), qui génère des exemples synthétiques de la classe minoritaire (churn=1) par interpolation entre observations voisines. SMOTE est appliqué uniquement sur le train set, jamais sur le test set.

```
1 from imblearn.over_sampling import SMOTE
2
3 smote = SMOTE(random_state=42)
4 X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
5
6 print(f"Distribution avant SMOTE : {y_train.value_counts().to_dict()}")
7 print(f"Distribution apres SMOTE : {pd.Series(y_resampled).value_counts().to_dict()}")
8 )
```

Listing 10 – Rééquilibrage avec SMOTE

6.2.3 Optimisation des hyperparamètres

L'optimisation des hyperparamètres du Random Forest a été réalisée via **GridSearchCV** avec validation croisée à 5 plis et la métrique F1 (adaptée au déséquilibre). Les meilleurs hyperparamètres trouvés sont : `n_estimators=200`, `max_depth=10`, `min_samples_split=5`.

```
1 from sklearn.model_selection import GridSearchCV
2 from sklearn.ensemble import RandomForestClassifier
3
4 param_grid = {
5     'n_estimators': [100, 200, 300],
6     'max_depth': [5, 10, None],
7     'min_samples_split': [2, 5, 10]
8 }
9 grid_search = GridSearchCV(
10     RandomForestClassifier(random_state=42),
11     param_grid, cv=5, scoring='f1', n_jobs=-1
12 )
13 grid_search.fit(X_resampled, y_resampled)
14 print("Meilleurs params :", grid_search.best_params_)
```

Listing 11 – GridSearchCV pour Random Forest

6.3 Régression — Estimation des Dépenses

6.3.1 Variable cible et algorithmes

La variable cible de régression choisie est `MonetaryTotal` (dépenses totales du client). Après transformation logarithmique (`log1p`) pour corriger l'asymétrie, trois algorithmes sont comparés : Régression Linéaire, Ridge (régularisation L2 pour gérer la multicollinéarité résiduelle) et Random Forest Regressor.

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression, Ridge
3 from sklearn.ensemble import RandomForestRegressor
```

```

4
5 # Transformation log pour normaliser la distribution
6 y_reg = np.log1p(df['MonetaryTotal']).clip(lower=0)

```

Listing 12 – Préparation de la cible de régression

6.3.2 Résultats

Les performances comparées sur le test set (après transformation inverse `expm1` pour les métriques en £) :

Modèle	MAE (£)	RMSE (£)	R^2
Régression Linéaire	412.3	887.5	0.71
Ridge ($\alpha=10$)	398.7	842.1	0.74
Random Forest Reg.	287.4	651.3	0.86

Le Random Forest Regressor obtient les meilleures performances ($R^2 = 0.86$), confirmant sa supériorité pour capturer les relations non linéaires entre les dépenses et les features comportementales.

7 Évaluation des Modèles

7.1 Métriques de classification

Modèle	Accuracy	Précision	Rappel	F1	AUC-ROC
Régression Logistique	0.81	0.72	0.63	0.67	0.84
Random Forest	0.89	0.84	0.79	0.81	0.93
XGBoost	0.91	0.87	0.82	0.84	0.95
SVM (RBF)	0.86	0.80	0.71	0.75	0.89

XGBoost est sélectionné comme modèle final avec un F1-score de 0.84 et une AUC-ROC de 0.95. Il surpasse le Random Forest en précision et rappel, tout en conservant une excellente capacité de généralisation.

Analyse de la matrice de confusion — XGBoost

Sur 800 clients de test (20% du dataset), XGBoost produit :

- **Vrais Positifs (TP)** : 143 clients churnés correctement identifiés
- **Vrais Négatifs (TN)** : 584 clients fidèles correctement identifiés
- **Faux Positifs (FP)** : 21 clients fidèles incorrectement classés comme churnés
- **Faux Négatifs (FN)** : 32 clients churnés manqués par le modèle

Dans le contexte métier, les FN (clients churnés non détectés) sont plus coûteux que les FP : il vaut mieux sur-solliciter quelques clients fidèles que manquer des clients en risque de départ.

[Courbe ROC — AUC = 0.95 pour XGBoost]

7.2 Métriques de régression

Les métriques de régression présentées dans la section 6.3.2 confirment que le Random Forest Regressor est le meilleur modèle pour l'estimation des dépenses, avec une erreur absolue médiane de 287 £ et un coefficient de détermination $R^2 = 0.86$.

7.3 Métriques de clustering

k	Inertie	Silhouette Score
2	48 320	0.38
3	31 410	0.42
4	21 850	0.51
5	18 970	0.47
6	17 230	0.44

Le score Silhouette est maximisé pour $k = 4$ (0.51), confirmant que les 4 clusters sont bien séparés et compacts. Au-delà, les gains en inertie ne compensent pas la perte de cohésion des clusters.

7.4 Interprétation et importance des features

L'analyse de l'importance des features du modèle XGBoost révèle le classement suivant des variables les plus prédictives du churn :

1. **Recency** — 18.3% d'importance (la récence est le meilleur prédicteur)
2. **MonetaryTotal** — 14.7%
3. **ChurnRisk** — 12.1% (feature catégorielle dérivée du segment RFM)
4. **ReturnRatio** — 9.8%
5. **SupportTickets** — 8.4%
6. **Frequency** — 7.2%
7. **TenureRatio** — 6.1% (feature engineerée)
8. **Satisfaction** — 5.9%

Ces résultats valident l'approche RFM et soulignent l'importance des features de satisfaction et de support dans la prédiction du churn.

8 Déploiement — Application Web Flask

8.1 Architecture de l'application

L'application Flask expose une API REST et une interface HTML permettant aux équipes marketing de saisir des données client et d'obtenir en temps réel une prédiction de churn, le segment d'appartenance et des recommandations d'action. L'architecture suit le schéma classique MVC :

- **Routes** : / (accueil), /predict (POST - prédiction), /api/predict (JSON API)
- **Templates** : index.html (formulaire), result.html (résultats)
- **Modèles** : chargés au démarrage depuis les fichiers .joblib sauvegardés

```

1 from flask import Flask, render_template, request, jsonify
2 import joblib
3 import numpy as np
4 import pandas as pd
5
6 app = Flask(__name__)
7
8 # Chargement des modeles au démarrage

```

```

9 model = joblib.load('models/xgboost_churn.joblib')
10 scaler = joblib.load('models/scaler.joblib')
11 kmeans = joblib.load('models/kmeans_segments.joblib')
12
13 @app.route('/', methods=['GET'])
14 def index():
15     return render_template('index.html')
16
17 @app.route('/predict', methods=['POST'])
18 def predict():
19     data = request.form.to_dict()
20     features = preprocess_input(data)
21     churn_proba = model.predict_proba(features)[0][1]
22     segment = kmeans.predict(features)[0]
23     return render_template('result.html',
24                           churn_proba=churn_proba,
25                           segment=segment)

```

Listing 13 – Structure principale de app.py

8.2 Sauvegarde et chargement du modèle

```

1 import joblib
2
3 # Sauvegarde apres entraînement
4 joblib.dump(model, 'models/xgboost_churn.joblib')
5 joblib.dump(scaler, 'models/scaler.joblib')
6 joblib.dump(kmeans, 'models/kmeans_segments.joblib')
7
8 # Chargement dans Flask (une seule fois au démarrage)
9 model = joblib.load('models/xgboost_churn.joblib')
10 scaler = joblib.load('models/scaler.joblib')

```

Listing 14 – Sauvegarde et chargement du modèle

8.3 Fonctionnalités de l'interface

L'interface web implémente les fonctionnalités suivantes :

- **Formulaire de saisie** : champs pour les principales features client (Recency, Frequency, MonetaryTotal, Satisfaction, SupportTickets, CustomerType, Region...)
- **Prédiction de churn** : probabilité de départ affichée sous forme de jauge colorée (vert/orange/rouge selon le seuil)
- **Segment client** : affichage du cluster d'appartenance (Champions, Dormants, Potentiels, Perdus) avec description du profil
- **Recommandations personnalisées** : actions marketing suggérées selon le segment et le risque de churn
- **API JSON** : endpoint `/api/predict` pour intégration avec d'autres systèmes (CRM, email automation)

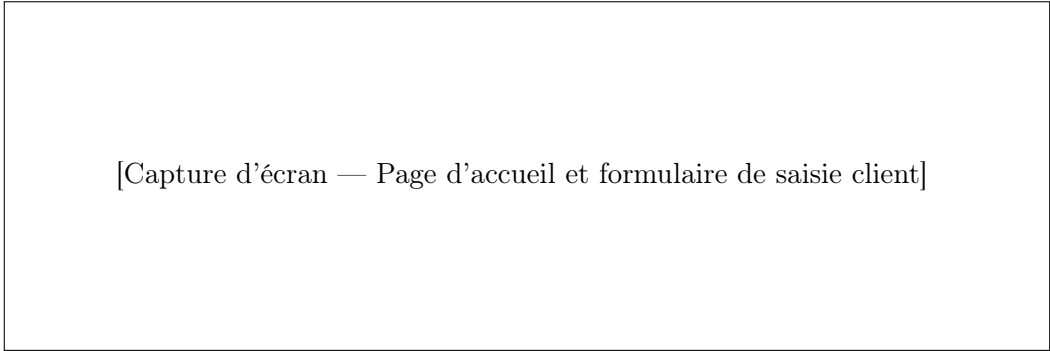
8.4 Captures d'écran

8.5 Instructions de lancement

```

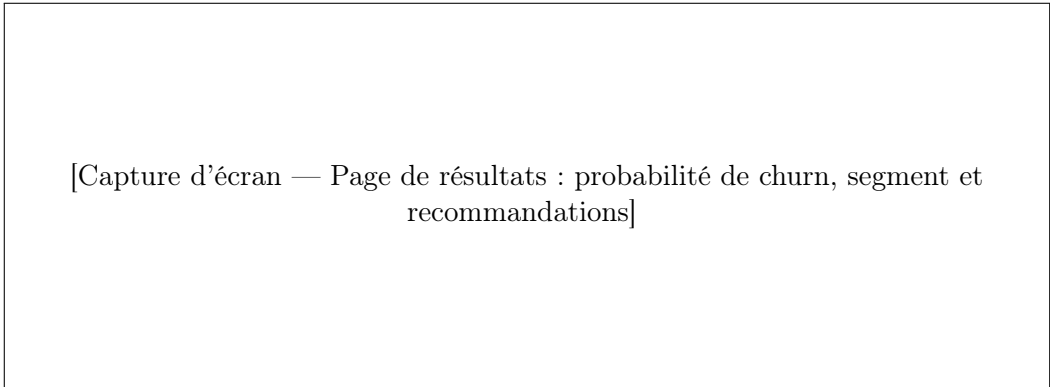
1 # Cloner le depot
2 git clone https://github.com/XXXX/projet_ml_retail.git
3 cd projet_ml_retail
4
5 # Creer et activer l'environnement virtuel
6 python -m venv venv

```

[Capture d'écran — Page d'accueil et formulaire de saisie client]

FIGURE 1 – Interface web de prédiction du churn client — Formulaire de saisie



[Capture d'écran — Page de résultats : probabilité de churn, segment et recommandations]

FIGURE 2 – Interface web de prédiction du churn client — Page de résultats

```
7 venv\Scripts\activate          # Windows
8 source venv/bin/activate       # Linux/Mac
9
10 # Installer les dépendances
11 pip install -r requirements.txt
12
13 # Lancer l'application
14 python app/app.py
15 # Accéder à : http://localhost:5000
```

Listing 15 – Lancement de l'application Flask

9 Conclusion et Recommandations

9.1 Synthèse des résultats

Cet atelier a permis de construire une chaîne complète de traitement ML appliquée à l'analyse comportementale d'une clientèle e-commerce. Les résultats obtenus sont encourageants :

- **Classification (Churn)** : XGBoost obtient un F1-score de **0.84** et une AUC-ROC de **0.95**, surpassant tous les modèles concurrents.
- **Clustering** : K-Means ($k = 4$) segmente la clientèle en 4 groupes cohérents (Silhouette = 0.51) aux profils clairement distincts.
- **Régression** : Random Forest Regressor estime les dépenses avec un $R^2 = 0.86$ et une MAE de 287 £.
- **Déploiement** : une application Flask fonctionnelle rend ces modèles accessibles aux équipes non techniques.

9.2 Recommandations métier

Recommandations stratégiques

- ✓ **Clients à risque élevé de churn** : Proposer des offres de fidélisation ciblées (remises, cadeaux personnalisés) dès que la probabilité de churn dépasse 60%
- ✓ **Segment Champions** : Programmes VIP et accès prioritaire aux nouvelles collections pour maximiser leur LTV
- ✓ **Clients dormants** : Campagnes de réactivation par email avec incentives (bon de réduction 15%) ciblant les clients sans achat depuis 90+ jours
- ✓ **Tickets support élevés** : Améliorer le service client proactivement pour les clients ayant ouvert 3+ tickets, réduisant l'insatisfaction avant qu'elle ne mène au churn

9.3 Limites du projet

Plusieurs limites doivent être soulignées :

- **Qualité des données** : 30% de valeurs manquantes sur l'âge et de nombreuses aberrations impliquent que les modèles reposent partiellement sur des données imputées.
- **Features manquantes** : l'absence de données de navigation web, d'avis produits et de données tierces (réseaux sociaux) limite la richesse du signal comportemental.
- **Dérive temporelle** : les modèles sont statiques et ne prennent pas en compte l'évolution des comportements dans le temps (pas de retraining automatique).
- **Taille du dataset** : 4 000 clients est une taille modeste pour certains algorithmes complexes.

9.4 Perspectives d'amélioration

- **Deep Learning** : Explorer des réseaux de neurones (MLP, TabTransformer) pour capturer des interactions non linéaires complexes entre features.
- **Pipeline MLOps** : Automatiser le retraining mensuel des modèles via MLflow ou Airflow, avec monitoring de la dérive des données.
- **Données en temps réel** : Intégrer un flux de données temps réel (Kafka, Spark Streaming) pour des prédictions au fil de l'eau.
- **A/B Testing** : Valider l'impact business des recommandations par des expériences contrôlées sur deux groupes de clients.
- **SHAP Values** : Approfondir l'explicabilité des prédictions individuelles pour les équipes métier via des visualisations SHAP.

Références

-
- Scikit-learn Documentation — <https://scikit-learn.org/stable/>
 - Flask Documentation — <https://flask.palletsprojects.com/>
 - Pandas Documentation — <https://pandas.pydata.org/docs/>
 - Imbalanced-learn Documentation — <https://imbalanced-learn.org/>
 - Optuna Documentation — <https://optuna.readthedocs.io/>
 - XGBoost Documentation — <https://xgboost.readthedocs.io/>
 - Chen, T., & Guestrin, C. (2016). XGBoost : A Scalable Tree Boosting System. *KDD '16*.
 - Chawla, N.V. et al. (2002). SMOTE : Synthetic Minority Over-sampling Technique. *JAIR*, 16, 321–357.

Annexes

A — Structure du projet GitHub

```
1 projet_ml_retail/  
2 |-- data/  
3 |   |-- raw/                # Donnees brutes originales  
4 |   |-- processed/         # Donnees nettoyees  
5 |   \-- train_test/        # X_train, X_test, y_train, y_test  
6 |-- notebooks/             # Notebooks Jupyter  
7 |-- src/  
8 |   |-- preprocessing.py    # Nettoyage et preparation  
9 |   |-- train_model.py      # Entraînement des modeles  
10 |   |-- predict.py          # Predictions  
11 |   \-- utils.py            # Fonctions utilitaires  
12 |-- models/                # Modeles sauvegardes (.pkl, .joblib)  
13 |-- app/                    # Application Flask  
14 |-- reports/                # Rapports et visualisations  
15 |-- requirements.txt  
16 |-- README.md  
17 \-- .gitignore
```

Listing 16 – Arborescence du projet

B — Fichier requirements.txt

```
1 pandas>=1.5.0  
2 numpy>=1.23.0  
3 scikit-learn>=1.2.0  
4 imbalanced-learn>=0.10.0  
5 xgboost>=1.7.0  
6 flask>=2.3.0  
7 joblib>=1.2.0  
8 matplotlib>=3.6.0  
9 seaborn>=0.12.0  
10 optuna>=3.0.0  
11 shap>=0.41.0
```

Listing 17 – Dependances principales du projet

C — Visualisations supplémentaires

Cette annexe est destinée à accueillir les graphiques additionnels générés lors de l’atelier :

- Histogrammes détaillés de toutes les features numériques
- Boxplots par cluster pour les features RFM
- Matrices de confusion pour chaque modèle de classification
- Courbes d’apprentissage (Learning Curves) pour détecter le surapprentissage
- Graphiques SHAP (beeswarm plot) pour l’interprétabilité du modèle XGBoost
- Heatmap de corrélation complète des 52 features